

CHƯƠNG 2

Không gian trạng thái và Phương pháp tìm kiếm



PGS.TS. Nguyễn Đình Hiền

Nội dung

- ***Vấn đề, giải quyết vấn đề***
- ***Khái niệm về thuật toán, thuật giải***
- ***Các nguyên lý của thuật giải heuristic***
- ***Các chiến lược tìm kiếm và Thuật giải A****



Vấn đề?

Những vướng mắc khó khăn cần giải quyết

Một yêu cầu tìm kiếm xử lý trong một ngữ cảnh cụ thể

Bao gồm:

- các sự kiện
- các thông tin
- những ràng buộc nhất định.



vấn đề = bài toán

Mô hình vấn đề

$$A \rightarrow B$$

A: giả thiết, điều kiện ban đầu

B: kết luận cần đạt đến

$\rightarrow$: suy luận hay giải pháp cần xác định =
một số hữu hạn bước



Phân loại vấn đề

- Xác định rõ
 - A, B đều rõ
- Chưa rõ
 - A rõ, B chưa rõ
 - A chưa rõ, B rõ
 - A, B đều chưa rõ



Thuật toán

- Thuật toán là giải pháp viết dưới dạng thủ tục và thỏa 3 tiêu chuẩn

Xác định : không mập mờ và có thể thực thi được

Hữu hạn

Đúng

➔ Thuật toán là một dãy hữu hạn các bước không mập mờ và có thể thực thi được, quá trình hành động theo các bước này phải dừng và cho kết quả mong muốn.

Thuật toán

- Tính tổng các số nguyên dương lẻ từ $1 \rightarrow n$
 - B1: $S=0$;
 - B2: $i=1$
 - B3: Nếu $i > n$ thì sang bước 7, ngược lại sang bước 4
 - B4: $S=S+i$
 - B5: $i=i+2$
 - B6: Quay lại 3
 - B7: Tổng cần tìm là S



Thuật toán

Thuật toán có thể được thể hiện qua:

Ngôn ngữ tự nhiên

Lưu đồ

Mã giả

NN lập trình

Ngoài ra thuật toán còn phải đạt hiệu quả cao hay có độ phức tạp thấp



Thuật toán

$O(\log_2 n)$
 $O(n)$
 $O(n \log_2 n)$
 $O(n^2)$
 $O(n^k)$

} polyminimal complexity → acceptable

$O(2^n)$
 $n!$

} high (Not polyminimal) complexity → unacceptable

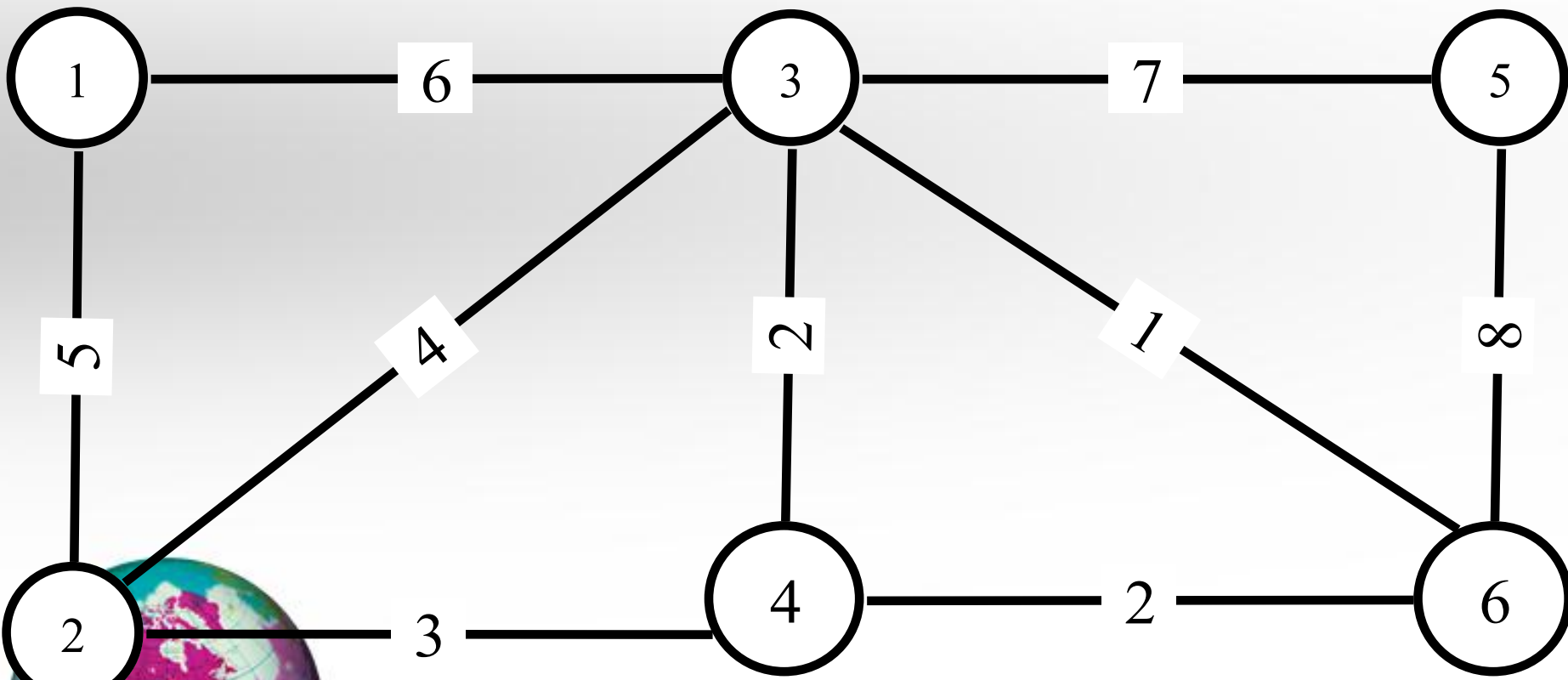


Ví dụ

- Có 1 trạm cấp nước và N điểm dân cư. Hãy xây dựng chương trình thiết kế tuyến đường ống nước cung cấp đến mọi nhà sao cho tổng chiều dài đường ống phải dùng là ít nhất. Giả sử rằng các đường ống chỉ được nối giữa 2 điểm dân cư hoặc giữa trạm cấp nước với điểm dân cư.



Ví dụ



Ví dụ

- Sắp xếp các cạnh theo thứ tự tăng của trọng số
- `for(i=0;i<n;i++)`
lấy cạnh e_i nhỏ nhất thêm vào T sao cho T không có chu trình
- Kết thúc ta có T là cây khung nhỏ nhất



Bài toán tìm kiếm

Vấn đề = Tìm kiếm mục tiêu



Không gian trạng thái

- Không gian trạng thái: tập tất cả các trạng thái **có thể có** của bài toán.



Các ví dụ

2	8	3
1	6	4
7		5



1	2	3
8		4
7	6	5



-
- Bài toán đổ nước
 - Bài toán giải tam giác
 - Bài toán người quỉ qua sông



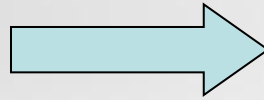
Không gian trạng thái

- Giữa các trạng thái có sự liên hệ gọi là chuyển trạng thái hay toán tử chuyển trạng thái.



Ví dụ

2	8	3
1	6	4
7		5



1	2	3
8		4
7	6	5

- Trạng thái đầu và cuối của bài toán 8 số
- Các toán tử: qua trái, phải, lên trên, xuống dưới



- Biểu diễn trạng thái bằng **(a, b, k)**

Với a, b là số người và quỳ ở bên bờ phải

k=1 là thuyền ở bờ phải, k=0 thuyền ở bờ trái

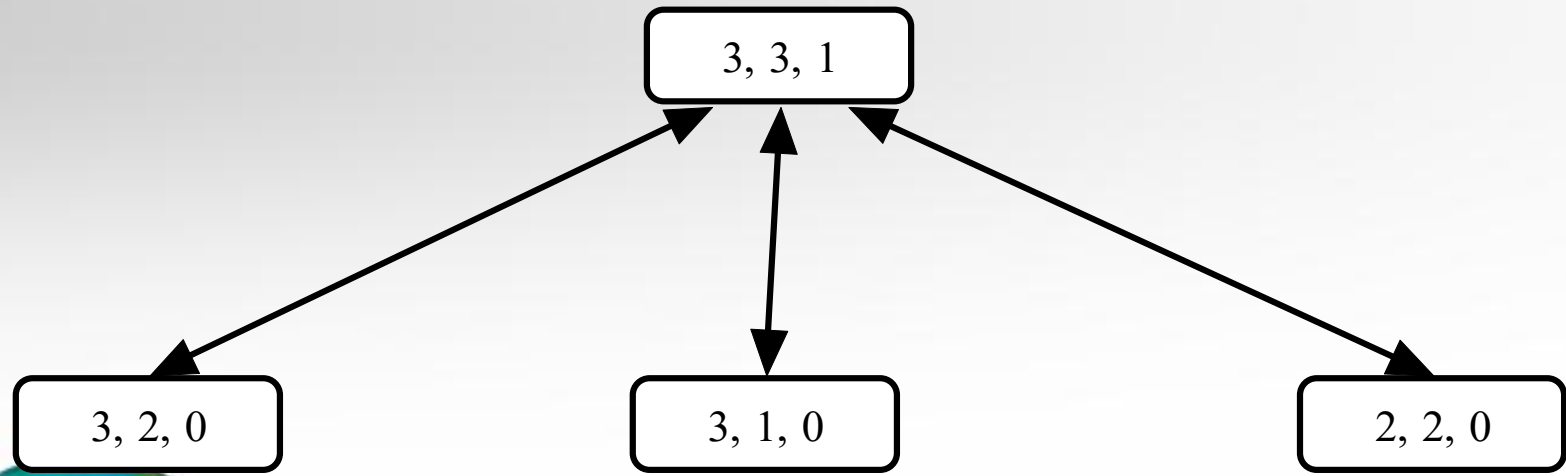
Không gian trạng thái

Trạng thái ban đầu (3, 3, 1)

Các toán tử: chở 1 người, 1 quỳ, 2 người, 2 quỳ,
1 người và 1 quỳ

Trạng thái kết thúc (0, 0, 0)



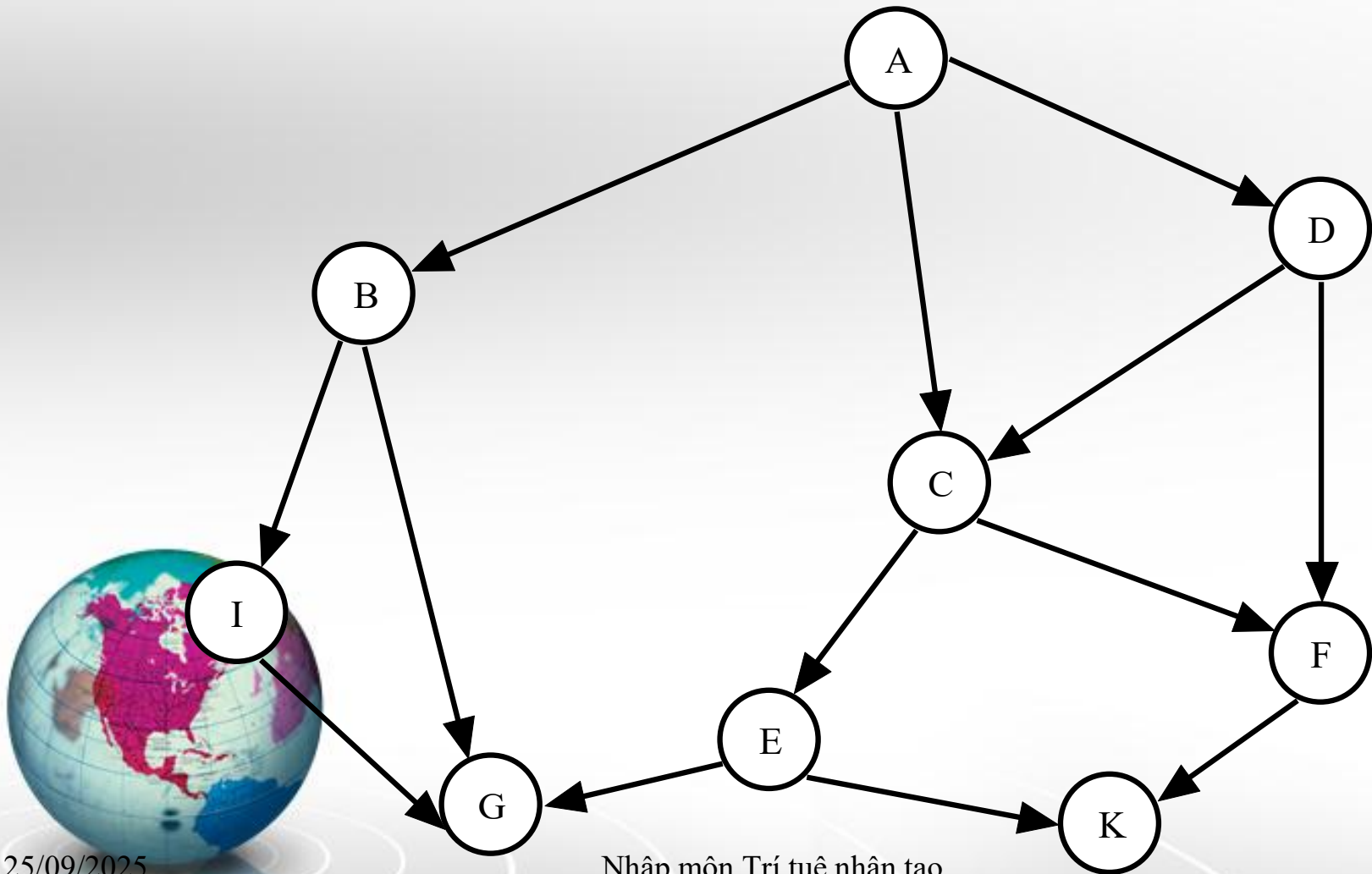


Không gian trạng thái

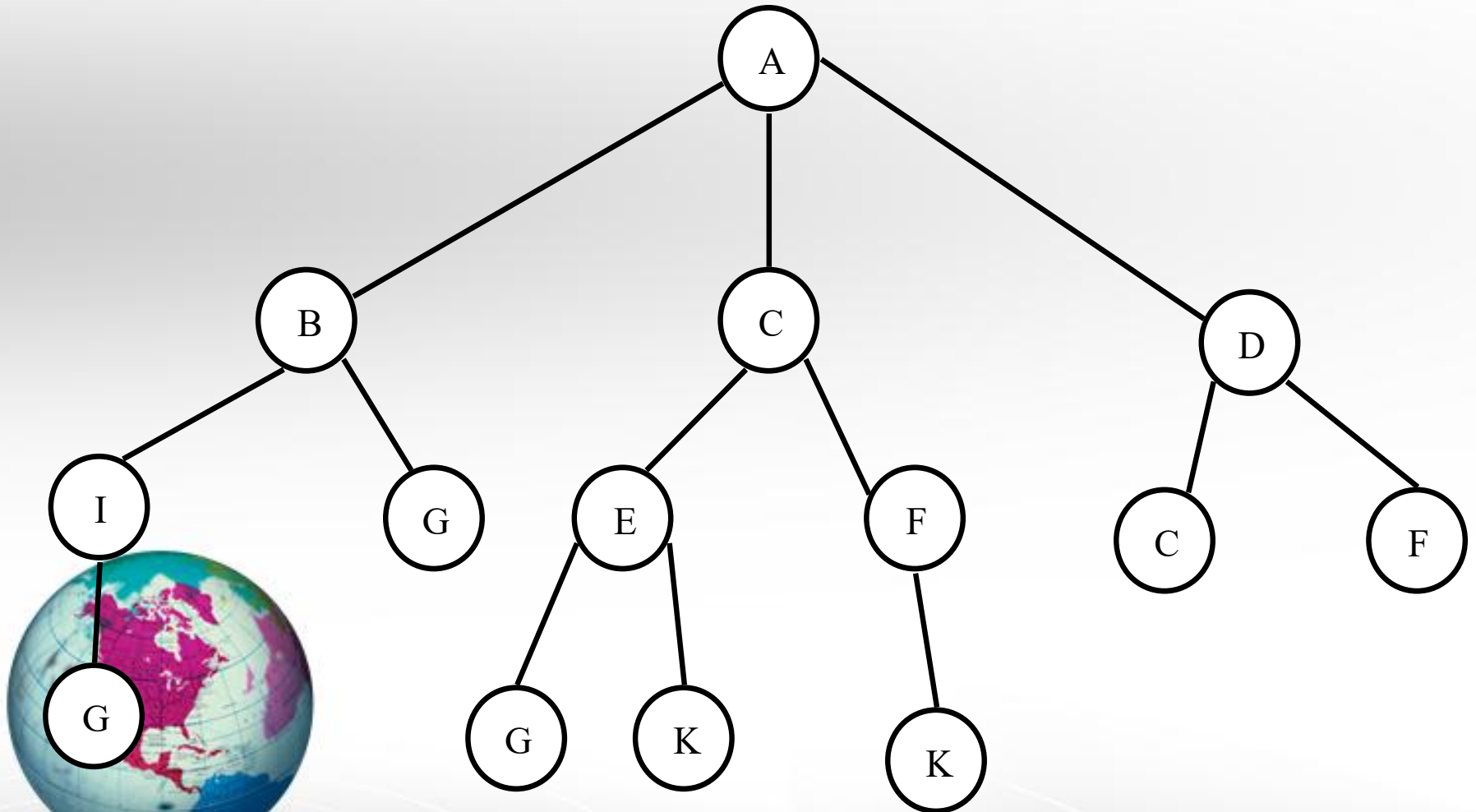
- Không gian trạng thái này có thể được biểu diễn bằng đồ thị
- Đồ thị trạng thái xác định khi:
 - Biết trạng thái đầu
 - Biết hàm $\text{next}(S)$ (tập hợp toán tử) trả về các trạng thái kế của S .
 - Biết trạng thái kết thúc (tập trạng thái kết thúc)



Cây tìm kiếm



Cây tìm kiếm



Phát biểu bài toán

- Trên một đồ thị có trọng số dương ta muốn tìm đường đi ngắn nhất từ một đỉnh xuất phát S_0 đến một đỉnh mục tiêu G .
- Giả sử ta có thêm thông tin như sau: tại mỗi đỉnh S ta biết ước tính quãng đường từ S đến mục tiêu là $h(S)$.



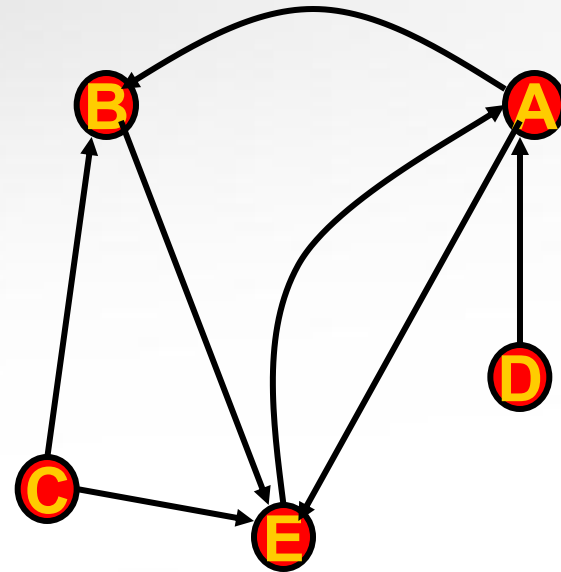
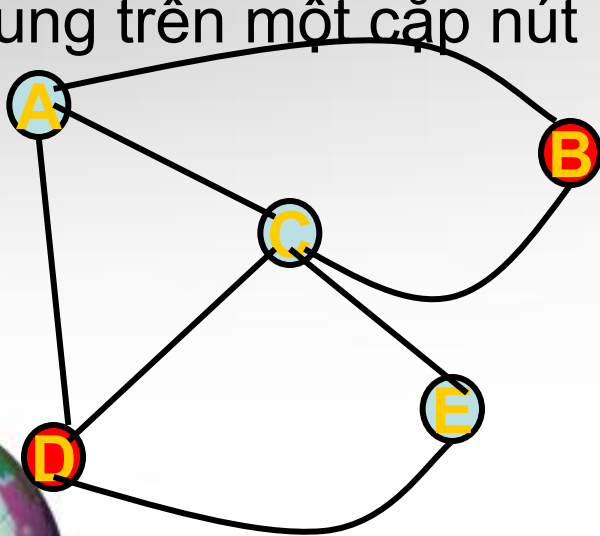
Các vấn đề trong thiết kế các CT tìm kiếm

- Xác định hướng tìm (forward hay backward reasoning).
- Cách lựa chọn luật để áp dụng (matching)
- Cách biểu diễn NODE của quá trình tìm (knowledge representation problem, frame problem).
- Các NODE trong đồ thị có thể được phát sinh nhiều lần, và có thể đã được xem xét trước đó trong quá trình duyệt → cần loại bỏ những NODE lặp lại. → cần lưu lại các NODE đã xét.



Lý thuyết đồ thị - Review

- Đồ thị: là một cấu trúc bao gồm:
 - Tập các nút $N1, N2, \dots, Nn, \dots$ Không hạn chế
 - Tập các cung nối các cặp nút, có thể có nhiều cung trên một cặp nút



Nút: {A,B,C,D,E}

Cung: {(a,d), (a,b), (a,c), (b,c), (c,d), (c,e), (d,e)}



Đặc tính đồ thị

- **Đồ thị có hướng:** là đồ thị với các cung có định hướng, nghĩa là cặp nút có quan hệ thứ tự trước sau **theo từng cung**. Cung (N_i, N_j) có hướng từ N_i đến N_j , Khi đó N_i là nút cha và N_j là nút con.
- **Nút lá:** là nút không có nút con.
- **Path:** là chuỗi có thứ tự các nút mà 2 nút kế tiếp nhau tồn tại một cung.



Đặc tính đồ thị

- **Đồ thị có gốc:** Trên đồ thị tồn tại nút X sao cho tất cả các path đều đi qua nút đó. X là gốc - Root
- **Vòng :** là một path đi qua nút nhiều hơn một lần
- **Cây:** là graph mà không có path vòng
- **Hai nút nối nhau :** nếu có một path đi qua 2 nút đó



Các chiến lược tìm kiếm

- Tìm kiếm mù
- Tìm kiếm tốt nhất
- Tìm kiếm heuristic

→ Mục tiêu: tìm ra một solution path
và/hay solution path tốt nhất



Tìm kiếm mù

- Tìm kiếm theo chiều sâu
- Tìm kiếm theo chiều rộng
- Tìm kiếm sâu dần



Tìm kiếm theo chiều sâu

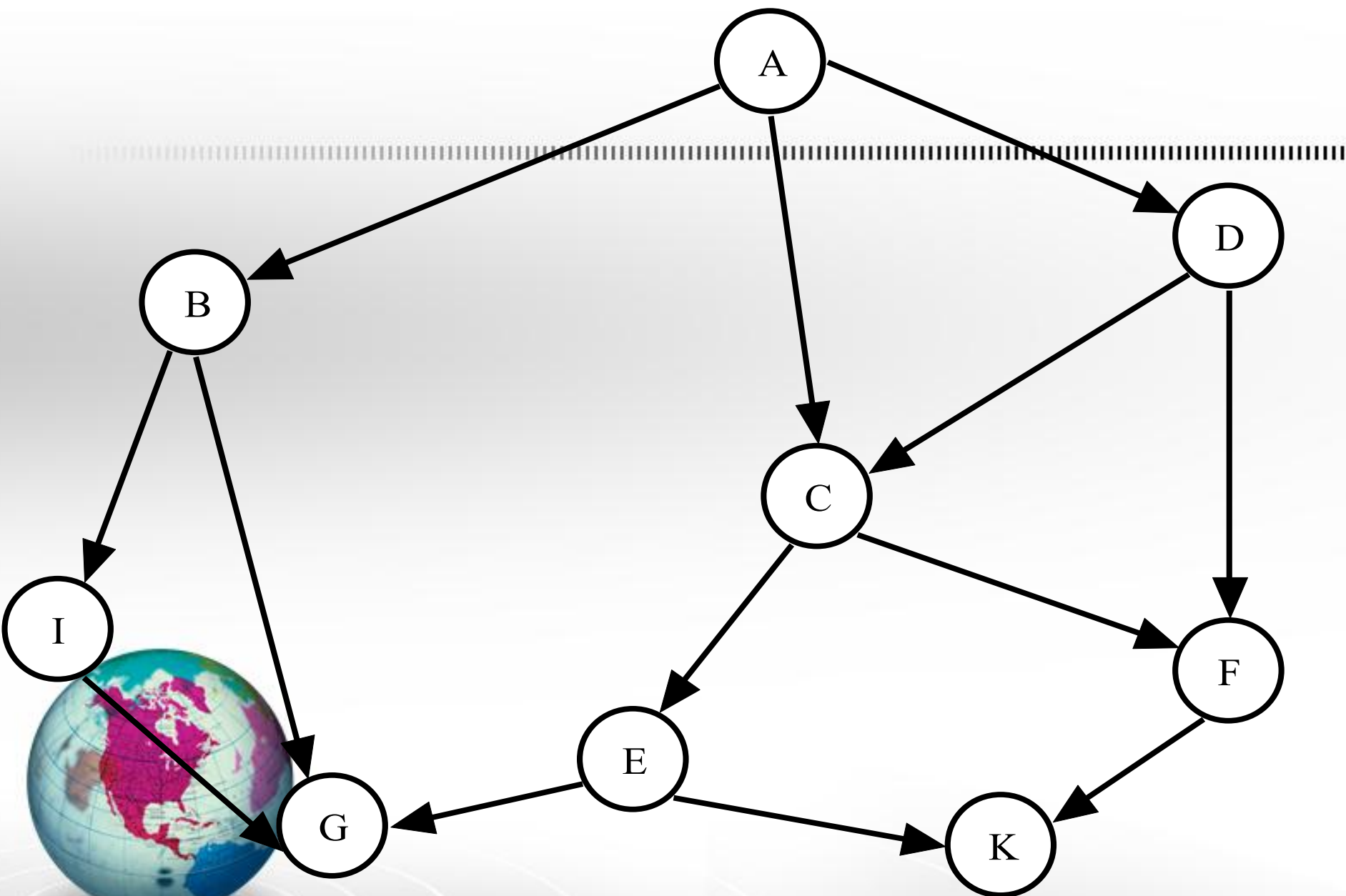
- Tìm kiếm sâu trong không gian bài toán được bắt đầu từ một nút rồi tiếp tục cho đến khi hoặc đến ngõ cụt hoặc đến đích
- Tại mỗi nút có luật trọng tài

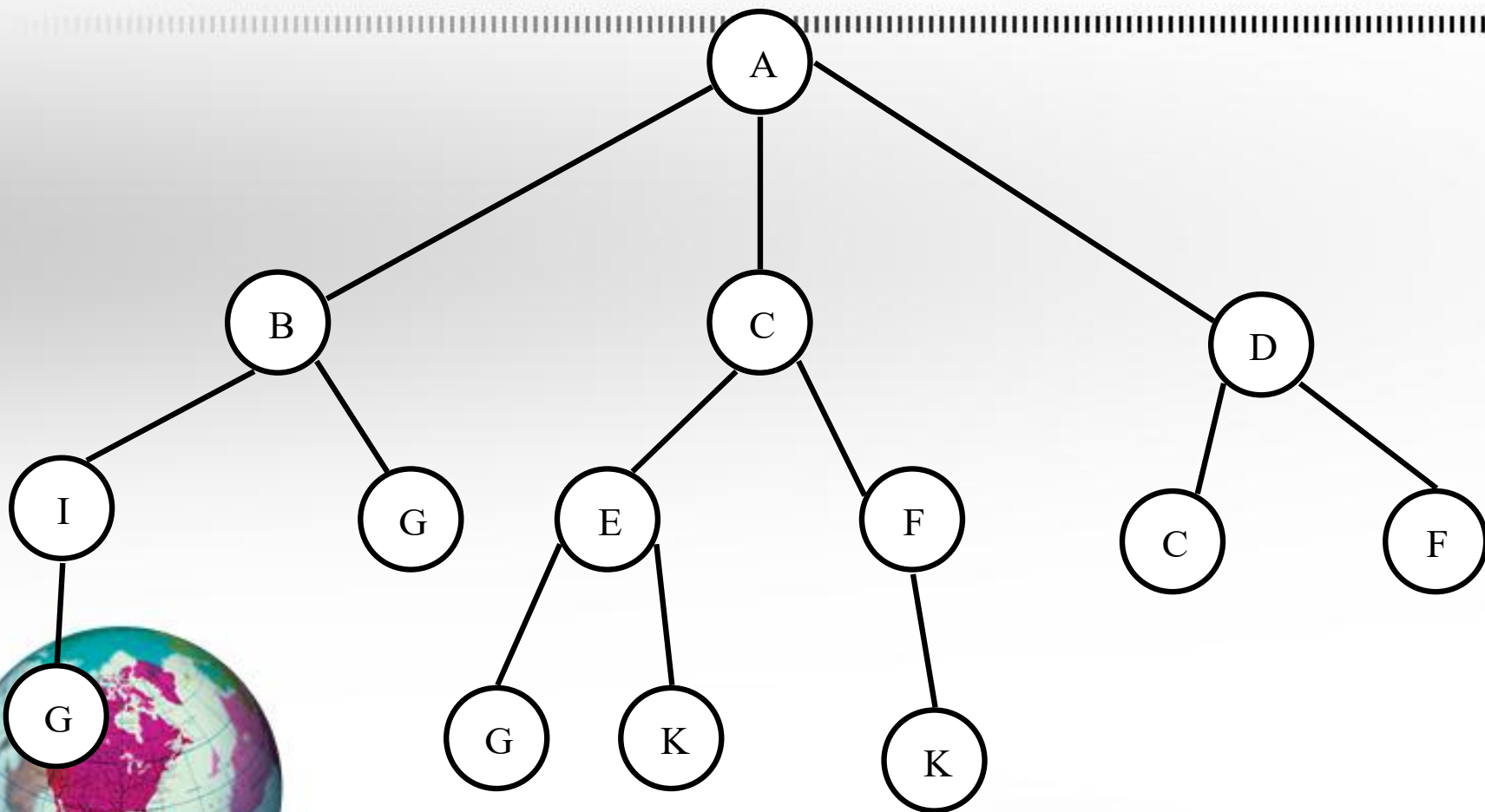


Tìm kiếm theo chiều sâu

- $O = [S]; C = \{\};$
- While O khác rỗng
- {
 - Lấy p từ đầu O
 - Duyệt p
 - Bỏ p vào C
 - Với mỗi q kề p , q không thuộc C : bỏ q vào đầu O
- }







Tìm kiếm theo chiều rộng

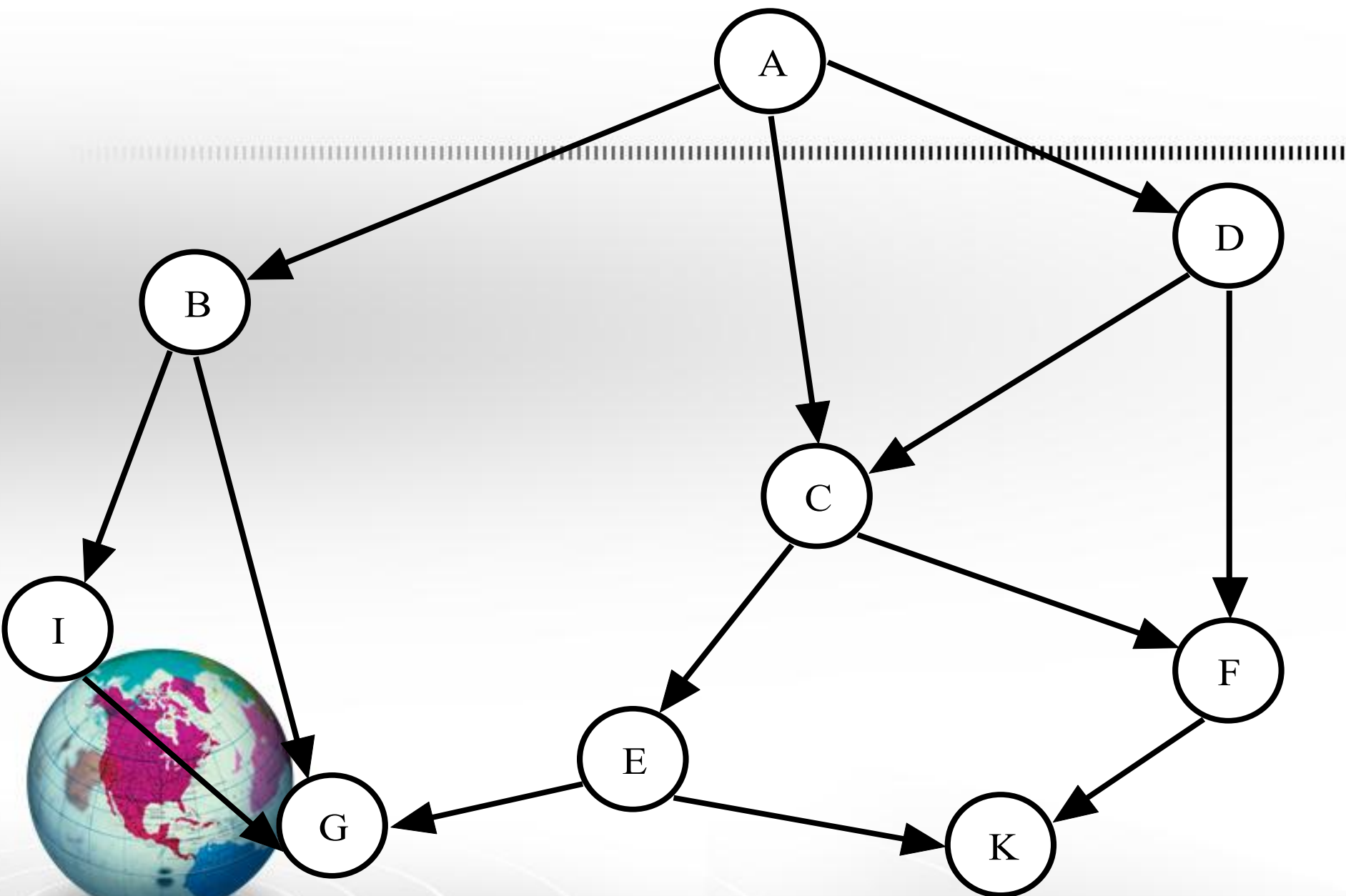
- Tìm kiếm trên tất cả các nút của một mức trong không gian bài toán trước khi chuyển sang các nút của mức tiếp theo

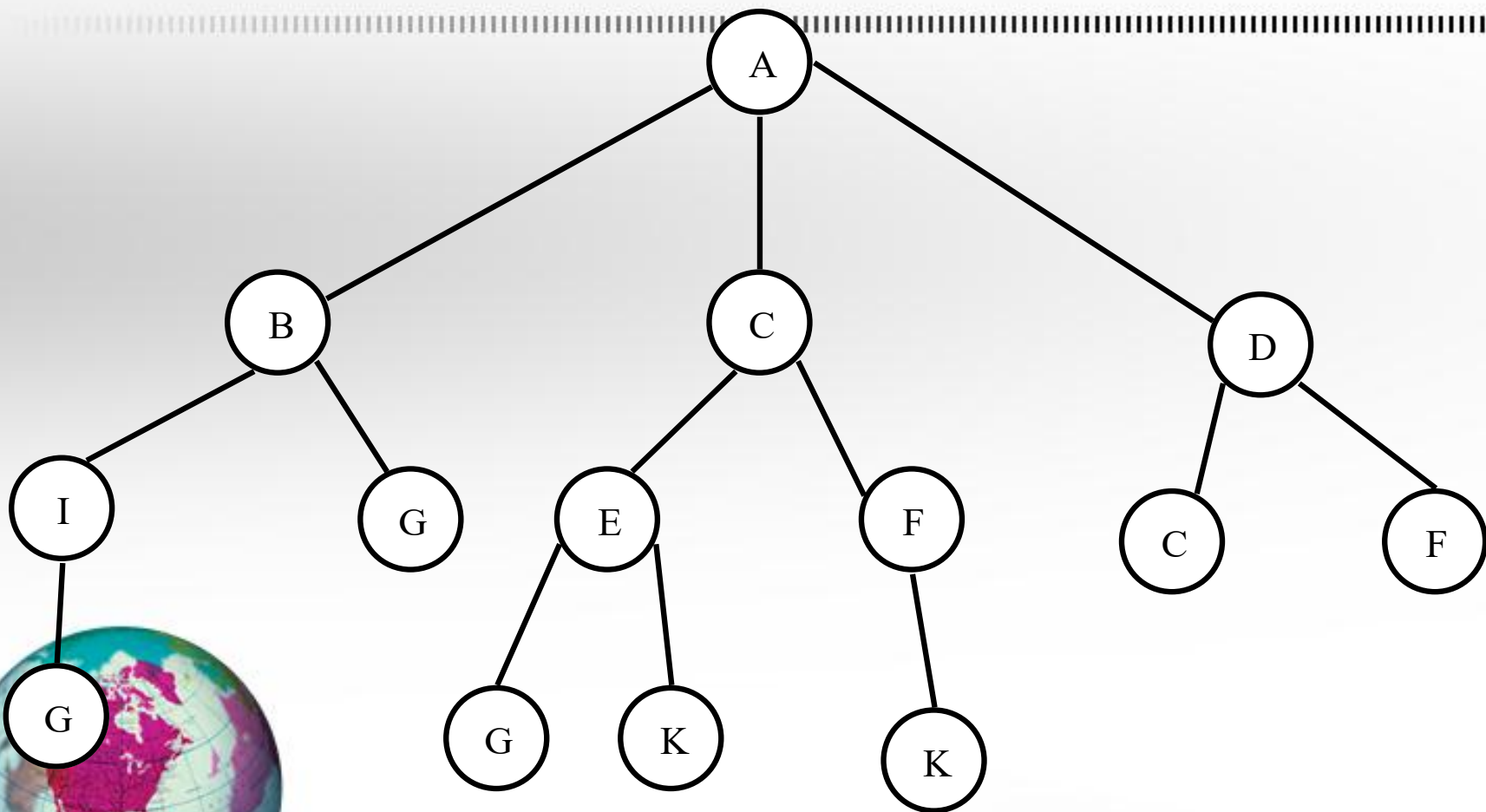


Tìm kiếm theo chiều rộng

- $O = [S]; C = \{\}$;
- While O khác rỗng
- {
 - Lấy p từ đầu O
 - Duyệt p
 - Bỏ p vào C
 - Với mỗi q kề p , q không thuộc C : bỏ q vào cuối O
- }







- **Bài tập 3.** Đại dương được xem như là một mặt phẳng toạ độ trên đó có n hòn đảo với toạ độ lần lượt là $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. Một chiếc ca nô xuất phát từ đảo d_1 muốn tuần tra đến đảo d_2 . bình xăng của ca nô chỉ chứa đủ xăng để đi được một quãng đường dài không quá m (km). Trên đường đi ca nô có thể ghé một số đảo nào đó để tiếp thêm xăng, lúc này ca nô được tiếp thêm xăng đầy bình chứa. Hãy chỉ ra một đường đi từ đảo d_1 đến đảo d_2 sao cho số lần ghé đảo trung gian để tiếp thêm xăng là ít nhất.

Tìm kiếm sâu dần

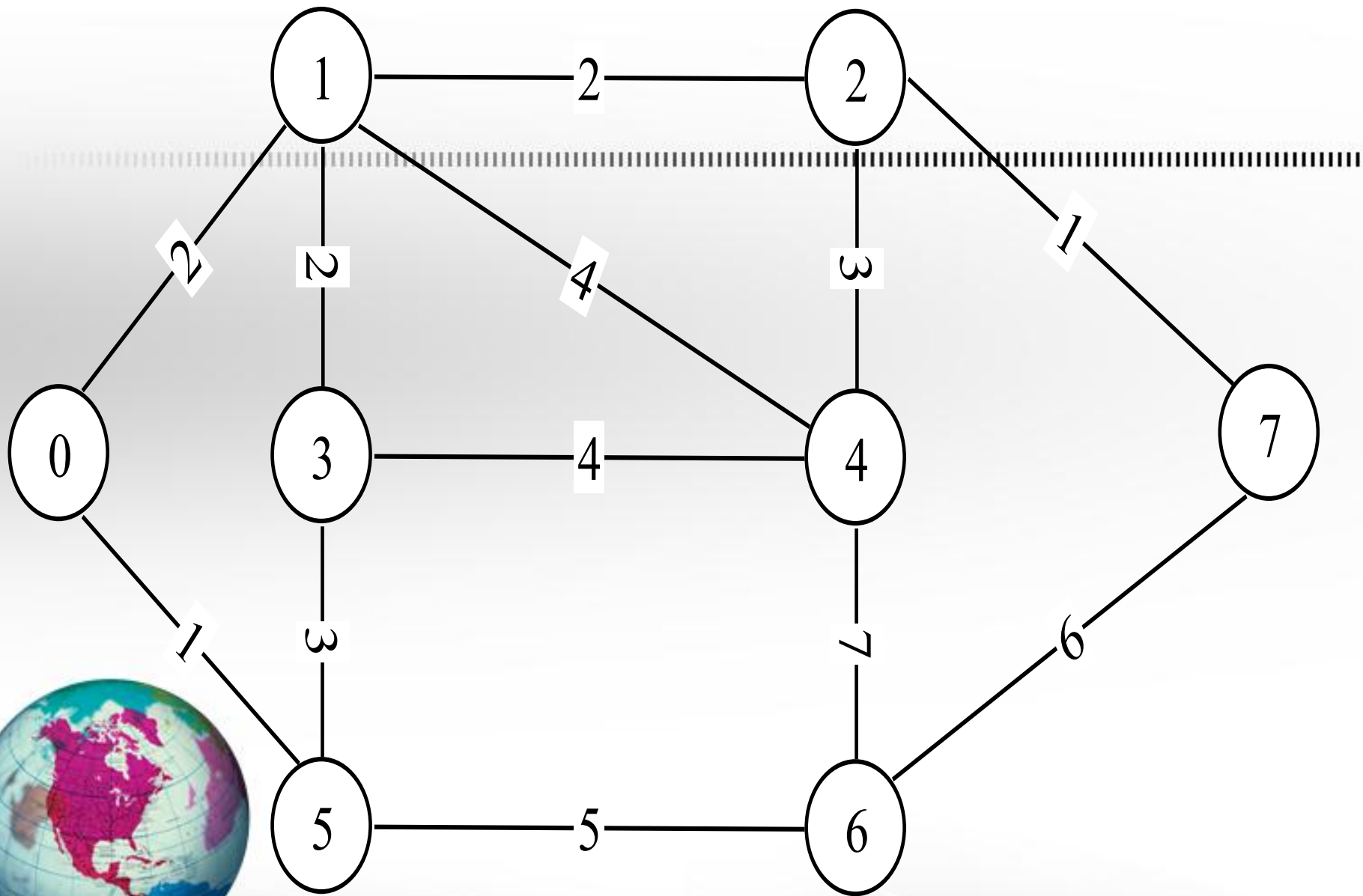
- Kỹ thuật tìm kiếm sâu dần là thực hiện việc tìm kiếm với độ sâu ở mức giới hạn d nào đó. Nếu không tìm ra nghiệm ta tăng độ sâu lên $d+1$ và lại tìm kiếm theo độ sâu tới mức $d+1$. Quá trình trên được lặp lại với d lần lượt là $1, 2, \dots$ đến độ sâu max nào đó

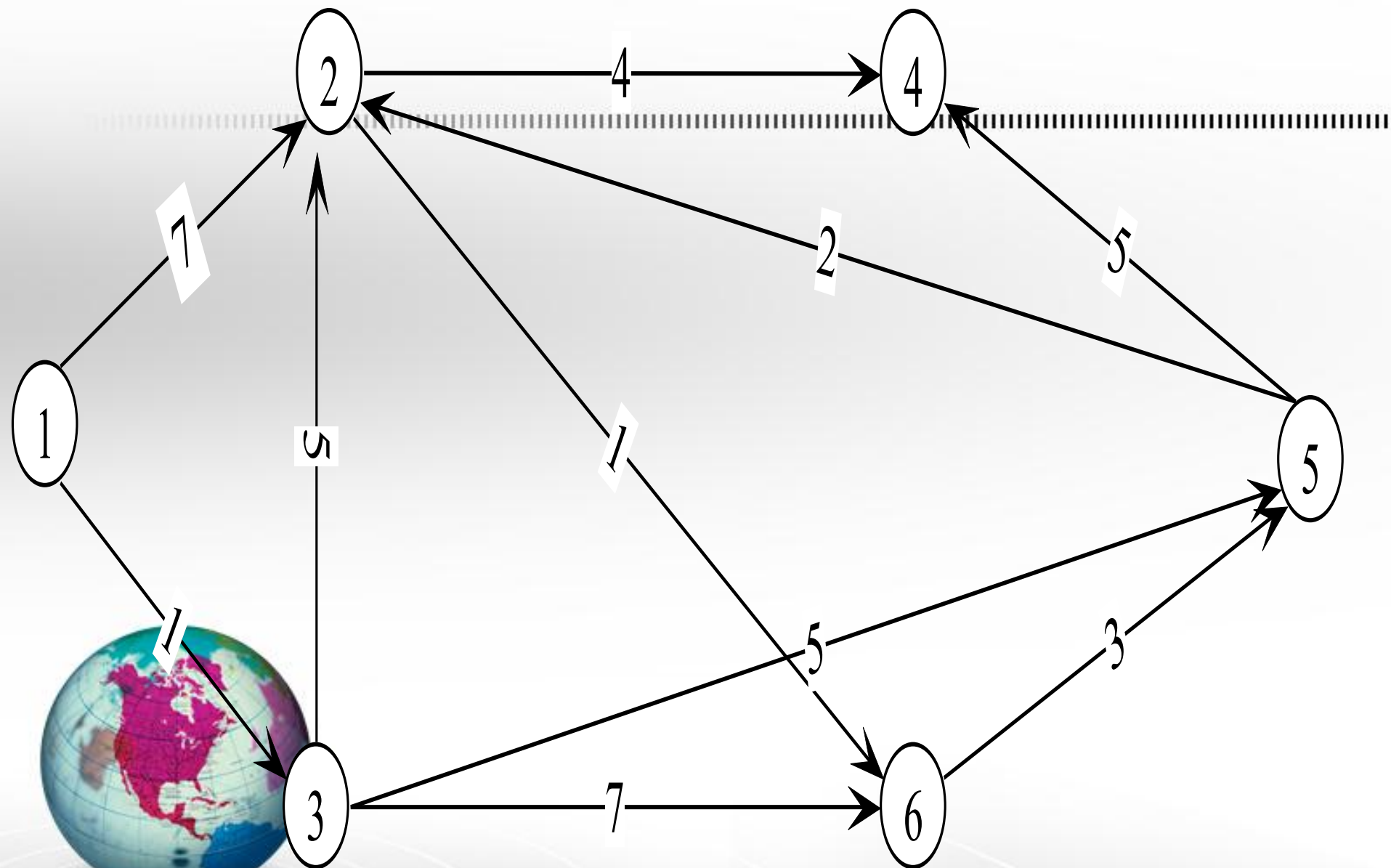


Tìm kiếm tốt nhất

- Dùng tri thức về bài toán để hướng dẫn
- Tại mỗi nút được xem xét: tìm kiếm tiếp tục theo nhánh nào tin tưởng sẽ dẫn đến lời giải.







Tìm kiếm đường đi có giá thành cực tiểu

Thuật toán Dijkstra (1)

Mỗi đỉnh tương ứng với 1 số $g(n)$: giá thành đi từ đỉnh ban đầu tới n

C : đỉnh đóng

O : đỉnh mở

Bước 1: $O = \{S\}$

$C = \{\}$

$g(S) = 0$

Bước 2: **While ($O \neq \{\}$)**

2.1 Chọn N thuộc O có $g(N)$ nhỏ nhất

N : mục tiêu $\rightarrow$ dừng, thông báo kết quả

Nếu không tồn tại $N \rightarrow$ dừng

2.2 Chuyển N qua C , và mở các Q sau N

Bước 3: Không có kết quả.



Tìm kiếm đường đi có giá thành cực tiểu

Thuật toán Dijkstra (2)

- **2.2 Chuyển N qua C, và mở các Q sau N**

2.2.1 Nếu Q đã có trong O

nếu $g(Q) > g(N) + \text{cost}(N, Q)$

$g(Q) = g(N) + \text{cost}(N, Q)$

$\text{prev}(Q) = N$

2.2.2 Nếu Q chưa có trong O

$g(Q) = g(N) + \text{cost}(N, Q)$

$\text{prev}(Q) = N$



Tìm kiếm cực tiểu sử dụng hàm đánh giá - Thuật toán A*

Cụ thể trong quá trình lựa chọn đỉnh để duyệt xét các đỉnh kế tiếp thì thuật giải A* dựa vào giá trị sau:

$$f(N) = g(N) + h(N)$$

với $g(N)$ số đo lộ trình từ S tới N

$f(N)$ ước tính độ dài từ S đến N



Tìm kiếm cực tiểu sử dụng hàm đánh giá - Thuật toán A*

Bước 1:

$$C = \{\};$$

$$O = \{S\};$$

$$g(S) = 0;$$

$$f(S) = h(S);$$



Tìm kiếm cực tiểu sử dụng hàm đánh giá - Thuật toán A*

Bước 2:

```
while( $O \neq \{\}$ )  
{
```

2.1 Chọn $N \in O$ có $f(N)$ nhỏ nhất

2.2 Lấy N từ O cho vào C

2.3 if($N \equiv G$)

Dừng. Kết luận: tìm được



Tìm kiếm cực tiểu sử dụng hàm đánh giá - Thuật toán A*

2.4 Xét các đỉnh kế Q của N

TH1: $Q \in O$

if($g(Q) > g(N) + w(N, Q)$)

$g(Q) = g(N) + w(N, Q);$

$f(Q) = g(Q) + h(Q);$

$prev(Q) = N$

TH2: $Q \notin O$

$g(Q) = g(N) + w(N, Q);$

$f(Q) = g(Q) + h(Q);$

$prev(Q) = N$



Bước 3: Kết luận...

A* - Ví dụ

2	8	3
1	6	4
7		5



1	2	3
8		4
7	6	5

- Trạng thái đầu và cuối của bài toán 8 số
 - Các toán tử: qua trái, phải, lên trên, xuống dưới
- $h(S_i)$ = số nút đúng của S_i so với G**





25/09/2025

8	1		1
4	3	2	5
7	6	5	6

8	3	1	2
	4	2	5
7	6	5	7

8		1	0
4	3	2	5
7	6	5	5

8	3	1	1
4		2	5
7	6	5	6

8	3	1	2
4	2		5
7	6	5	7

8	3	1	2
4	6	2	6
7		5	8

	8	1	1
4	3	2	5
7	6	5	6

nhập môn trí tuệ nhân tạo

Puzzle – Cài đặt

```
#define UP 1  
#define DN 2  
#define LT 3  
#define RT 4  
#define NO 0
```



Puzzle – Cài đặt

```
typedef char TAB[3][3];  
struct info{  
    TAB S;  
    unsigned g,h,f;  
    int d;  
};
```



Puzzle – Cài đặt

```
define SIZE 500
typedef struct List{
    int n;
    info e[SIZE];
    List(){n=0;}
    void them(info X);
    info timfnho();
    void xuatlist();
    int vitri(info X)
};
```



Puzzle – Cài đặt

```
List O,C,KQ;  
TAB S0, G;  
void main()  
{  
    nhap();  
    if(Astart())  
    {  
        timkq();  
        KQ.xuatlist();  
    }  
}
```



Puzzle – Cài đặt

```
void nhap();  
unsigned count(TAB S); //h  
int sobang(TAB S1, TAB S2);  
void ganbang(TAB S1, TAB S2); //gán S2 cho S1  
void timotrong(TAB S, int &k, int &l)  
void bangke(TAB N, int d, TAB S,int k, int l) //S là bảng kề của N  
int Astart();  
void xulyke(info X, int d, TAB S);  
void xuất(TAB S);  
void timkq();
```



Puzzle – Cài đặt

```
int Astart()
```

```
{
```

```
    info X;
```

```
    ganbang(X.S,S0);
```

```
    X.g=0;
```

```
    X.h=count(S0);
```

```
    X.f=X.h;
```

```
    X.d=NO;
```

```
    O.them(X);
```

```
    while(O.n>0)
```

```
    {
```

```
        X=O.timfnho();
```

```
        C.them(X);
```



Puzzle – Cài đặt

```
while(O.n>0)
{
    X=O.timfnho();
    C.them(X);
    if(sobang(X.S,G)) return 1;
    timotrong(X.S, k, l);
    if(k>0)
    {
        d=UP;
        bangke(X.S,d,BK,k,l);
        xulyke(X,d,BK);
    }
}
```



Puzzle – Cài đặt

```
if(k<2){...}  
if(l>0){...}  
if(l<2){...}  
}
```

```
return 0;
```

```
}
```



Puzzle – Cài đặt

```
void xulyke(info X, int d, TAB S)
```

```
{
```

```
    info Y;
```

```
    int k;
```

```
    if(C.vitri(S)==-1) return;
```

```
    if(k=O.vitri(S)==-1)
```

```
    {
```

```
        ganbang(Y.S,S);
```

```
        Y.g=X.g+1;
```

```
        Y.h=count(S);
```

```
        Y.f=Y.g+Y.h;
```

```
        Y.d=d;
```

```
        O.them(Y);
```

```
    }
```



Puzzle – Cài đặt

}

else if($X.g+1 < O.e[k].g$)

{

$O.e[k].g = X.g+1;$

$O.e[k].f = O.e[k].g + O.e[k].h;$

$O.e[k].d = d;$

}

}

